DESIGN AND ANALYSIS OF ALGORITHMS

Sessional Exam - Detailed Solutions

1. Explain asymptotic notations (Big-O, Big-Ω, Big-Θ) with examples.

Asymptotic notations are mathematical tools used to describe the limiting behavior of an algorithm's running time or space complexity as the input size (n) tends towards infinity. They help us classify algorithms according to their performance bounds.

1. Big-O Notation (O) - Upper Bound

Big-O notation describes the worst-case scenario. It provides an upper bound on the growth rate of a function, meaning the algorithm will never take more time than this bound.

- **Definition:** A function $f(n)$ is said to be $O(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq f(n) \leq c \cdot g(n) \text{ for all } n \geq n_0$$

- **Example:** Let $f(n) = 3n + 2$.
  We want to find $g(n)$ such that $3n + 2 \leq c \cdot g(n)$.
  For $n \geq 2$, $3n + 2 \leq 3n + n = 4n$.
  Here, $c = 4$ and $g(n) = n$.
  Therefore, $f(n) = O(n)$.

2. Big-Omega Notation (Ω) - Lower Bound

Big-Omega notation describes the best-case scenario. It provides a lower bound, meaning the algorithm will take at least this amount of time.

- **Definition:** A function $f(n)$ is said to be $\Omega(g(n))$ if there exist positive constants c and n_0 such that:

$$0 \leq c \cdot g(n) \leq f(n) \text{ for all } n \geq n_0$$

- **Example:** Let $f(n) = 3n + 2$.
  We want to find $g(n)$ such that $c \cdot g(n) \leq 3n + 2$.
  For all $n \geq 1$, $3n \leq 3n + 2$.

Here, $c = 3$ and $g(n) = n$.

Therefore, $f(n) = \Omega(n)$.

3. Big-Theta Notation (Θ) - Tight Bound

Big-Theta notation describes the average or exact bound. It is used when an algorithm has the same upper and lower bound growth rate.

- **Definition:** A function $f(n)$ is said to be $\Theta(g(n))$ if there exist positive constants c_1, c_2, and n_0 such that:

$$0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \text{ for all } n \geq n_0$$

- **Example:** Let $f(n) = 3n + 2$.

  From the previous examples, we proved that $3n \leq 3n + 2 \leq 4n$ for $n \geq 2$.

  Because $f(n)$ is both $O(n)$ and $\Omega(n)$, we can conclude that $f(n) = \Theta(n)$.

2. Solve the recurrence relation using substitution method: T(n) = 2T(n/2) + n

(Note: The substitution method is often applied via the back-substitution/iteration method to find the explicit pattern. Here is the step-by-step expansion).

Step 1: Write down the initial equation.

$$T(n) = 2T(n/2) + n \quad \text{--- (Equation 1)}$$

Step 2: Substitute to find the pattern.

To find $T(n/2)$, substitute $n/2$ into Equation 1:

$$T(n/2) = 2T(n/4) + n/2$$

Now, substitute this back into Equation 1:

$$T(n) = 2[2T(n/4) + n/2] + n$$

$$T(n) = 4T(n/4) + n + n$$

$$T(n) = 4T(n/4) + 2n \quad \text{--- (Equation 2)}$$

Next, find $T(n/4)$ by substituting $n/4$ into Equation 1:

$$T(n/4) = 2T(n/8) + n/4$$

Substitute this back into Equation 2:

$$T(n) = 4[2T(n/8) + n/4] + 2n$$

$$T(n) = 8T(n/8) + n + 2n$$

$$T(n) = 8T(n/8) + 3n \quad \text{--- (Equation 3)}$$

Step 3: Generalize the pattern for k iterations.

Looking at the pattern after 1, 2, and 3 steps, we can generalize the equation after k steps as:

$$T(n) = 2^k\, T(n/2^k) + kn$$

Step 4: Apply the base case.

Assume the base condition is $T(1) = 1$. The recurrence will reach the base case when:

$$n/2^k = 1 \;\rightarrow\; n = 2^k$$

Taking base-2 logarithm on both sides:

$$k = \log_2 n$$

Step 5: Substitute k back into the generalized equation.

$$T(n) = 2^{\log_2 n} \cdot T(1) + (\log_2 n) \cdot n$$

Since $2^{\log_2 n} = n$ and assuming $T(1) = O(1)$:

$$T(n) = n(1) + n \log_2 n$$

$$T(n) = n + n \log_2 n$$

Final Answer: Dominating term is $n \log_2 n$. Therefore, the time complexity is:

$$T(n) = O(n \log n)$$

3. Define algorithm with example.

Definition:
An algorithm is a finite, step-by-step sequence of unambiguous instructions designed to solve a specific problem or perform a computation. It takes a set of input values, processes them, and produces a corresponding output.

A good algorithm must possess the following five characteristics:

1. **Input:** It takes zero or more inputs.
2. **Output:** It produces at least one output.
3. **Definiteness:** Each instruction must be clear and unambiguous.
4. **Finiteness:** It must terminate after a finite number of steps.
5. **Effectiveness:** Every operation must be basic enough to be carried out exactly and in a finite amount of time.

Example of an Algorithm:
Problem: Find the maximum number in a given list of numbers.

```
Algorithm: Find_Maximum(A, n)
Where 'A' is the array of numbers and 'n' is the size of the array.

Step 1: Start.
Step 2: Read the array A and its size n.
Step 3: Initialize a variable max to the first element of the array.
        Let max = A[0].
Step 4: For each element A[i] from index 1 to n-1, do the following:
        If A[i] > max, then update max = A[i].
Step 5: Print or return the value of max.
Step 6: Stop.
```